

Intelligent Parallelization Prediction

1st Jackson Oleson
dept. Computer Science
New York University
New York City, USA
jo1449@nyu.edu

2nd Mihir Prajapati
dept. Computer Science
New York University
New York City, USA
mp6570@nyu.edu

3rd Yu Minematsu
dept. Computer science
New York University
New York City, USA
ym3143@nyu.edu

Abstract—Optimizing the performance of CUDA kernels is a critical challenge in high-performance computing, particularly for large-scale data processing and computationally intensive algorithms. This work presents a machine learning-based framework for predicting optimal block and thread configurations for CUDA kernels, utilizing core inputs such as problem size, block count, threads per block, and device specifications. By incorporating general problem characteristics that significantly impact kernel performance, the framework identifies critical patterns to enhance execution efficiency.

This study underscores the importance of integrating both hardware capabilities and computational patterns into CUDA optimization strategies, providing a scalable solution for diverse GPU workloads. Our approach demonstrates potential for broader application in adaptive GPU scheduling and automated performance tuning in heterogeneous computing environments.

Index Terms—auto-tuning, kernel optimization, machine learning.

I. INTRODUCTION

The rapid evolution of high-performance computing (HPC) has made Graphics Processing Units (GPUs) indispensable for solving large-scale computational problems. GPUs excel at parallel processing, enabling significant acceleration in domains ranging from scientific simulations to machine learning. However, achieving optimal performance in GPU applications remains a non-trivial task, largely due to the sensitivity of CUDA kernel performance to hardware specifications and problem-specific characteristics. Identifying the optimal parameter configuration is crucial to fully exploit the computational potential of GPUs.

Traditional approaches to CUDA kernel optimization often rely on manual tuning, heuristic rules, or empirical testing, which can be time-consuming and suboptimal. Furthermore, such methods typically fail to account for the diverse and intricate relationships between problem size, memory access patterns, and hardware constraints that influence kernel performance. As GPU architectures and workloads become increasingly complex, there is a growing need for automated, data-driven techniques to streamline this optimization process.

Other approaches use auto-tuning, which is the process of automatically searching for the optimal combination of parameter values. Because an exhaustive search over the parameter space is often infeasible, more efficient methods needed to be introduced to speed up the tuning process. This has given rise to various auto-tuning tools such as CLTune[1],

Kernel Tuner[2], Kernel Tuning Toolkit (KTT)[3], and Auto-Tuning Framework (ATF)[4]

A. Contribution

In this work, we propose a machine learning based approach to optimize kernel performance across various devices. We implement a variety of CUDA kernels to represent multiple problem types. We then test them on a wide range of problem sizes, block sizes, and grid sizes across four different NVIDIA devices and report the computation time. We then learn a regressive model on the data that we have collected to predict the optimal configuration of blocks and threads per block given the problem size and device specifications as input.

II. RELATED WORK

Auto-tuning has long been a strategy used for optimizing the performance of algorithms across various devices and in production-level libraries that rely on reproducibility in diverse environments. Dating back to the 1970's, researchers were using auto-tuning to optimize the parameters of digital controllers and to reduce the workload requirement of the skilled personnel that had to maintain them[5]. In 1982, Suzuki and Fukunishi used least-squares and simplex methods to optimize the control system parameters of nuclear power plants[6]. In 1997, PhiPAC was introduced to generate optimal C code for matrix multiply on the CPU[7].

Over the years with the increasing usage of machine learning, several researchers have utilized these methods along with statistical methods to predict the performance of parallel applications. For instance, [9] used machine learning to predict GPU speedup for OpenMP applications, while [10] leveraged regression models and principal component analysis for OpenCL kernel optimization. [11] compared analytical models and machine learning methods like support vector machines and random forests for predicting GPU application execution times, highlighting the adaptability of machine learning in heterogeneous environments.

[14] introduced a static analyzer for CUDA kernels that predicts performance without program execution, aiding autotuners like Orio in selecting optimal configurations. This approach reduces empirical search overhead while achieving competitive optimization results. [15] proposed a tuning model to optimize grid and block size parameters for GPU kernels.

Their approach, which considers GPU architecture and memory access patterns, significantly reduces the search space compared to exhaustive methods, enabling faster identification of optimal configurations. [12] developed KLARAPTOR, a tool for dynamically optimizing CUDA kernel launch parameters using rational programs. By combining compile-time analysis with runtime evaluation, it predicts optimal thread block configurations for each kernel invocation, achieving efficient, data-adaptive performance optimization [13] proposed a machine learning framework for automatically selecting thread block sizes by leveraging dynamic performance counters. Their approach focuses on identifying profitable configurations and analyzing performance anomalies due to block size variations. Unlike our method, which predicts optimal configurations using static features and problem characteristics, their work emphasizes runtime profiling and dynamic performance events.

These developments underline the potential of predictive frameworks to optimize GPU workloads. However, existing approaches often focus on specific hardware architectures or limited problem domains, leaving room for generalized and scalable solutions. Our work builds on these foundations by proposing a machine learning-based model that predicts optimal CUDA kernel configurations across diverse devices, incorporating problem size and hardware specifications for enhanced performance.

III. PROPOSED IDEA

In this study, we propose a problem-specific predictive framework for estimating CUDA kernel parameters. Optimizing the performance of a parallel program involves considering numerous parameters, which can be broadly categorized into three groups:

- **Problem Characteristics** - These can include but are not limited to problem size, memory access patterns, cache utilization and per-thread computations.
- **Device Specifications** - These are hardware constraints that are native to the specific machine. Cache size, clock rate (hz) and number of multiprocessors fall into this category.
- **Kernel Parameters** - Parameters such as block size and grid size.

Our framework focuses on the latter two—device specifications and kernel launch parameters—since problem characteristics are inherently fixed in a problem-specific scenario and thus play a less significant role. Device specifications can be fixed as well, determined by the access to various different hardware.

We found a variety of algorithms that would see a benefit from the usage of GPU hardware and implemented parallel versions using the CUDA framework. In each implementation allowing problem size, number of blocks, and threads per block to be specified as input parameters. for each algorithm, we collect a large amount of data by running the program for a wide range of configurations and record the execution time. Once this data is collected, we learn an XGBoost regression model to predict the optimal parameter configuration for a

given problem size by conducting a grid search over the parameter space. We evaluate our results against a set of test data and conduct an empirical validation of our results for a few example problem sizes by comparing against a validation set that we gathered for these few example problem sizes. As an added result, we study the importance of each input feature using XGBoost’s internal feature importance methods as well as conduct an external audit using SHAP (SHapley Additive exPlanations)[8].

A. Data Collection

To collect the appropriate data for our experiments, we ran each program with problem sizes spanning many orders of magnitude (i.e. 100k, 1m, 10m, 100m...) For each problem size, we launched the kernel with a subset of grid sizes:

$$\{1, 2, 2^2, \dots, 2^{15}\}$$

and block sizes:

$$\left\{ \begin{array}{ccccc} 64, & 100, & 128, & 200, & 256, \\ 400, & 512, & 750, & 800, & 1024 \end{array} \right\}$$

Shell scripts run these iterations and output a *csv* file containing the kernel launch parameters as well as the device specifications.

Problems like 2D-Convolution, K-Means, and Matrix Multiplication, require extra configuration parameters: kernel size, number of clusters, and tile width respectively. For each of the problem sizes and parameter configurations above, we also iterate over the following kernel sizes:

$$\{3, 5, 7, 9\}$$

for 2D-CONV, the following cluster numbers:

$$\{5, 10, 50, 100, 500\}$$

for K-Means, and the following tile widths:

$$\{8, 16, 24, 32, 50, 64, 90, 128, 150, 192\}$$

for matrix multiplication. For our evaluations and predicted best configuration, we fix these parameters as well as the problem size to focus solely on the kernel parameters.

B. Experiment Settings

We analyzed a variety of kernels representing diverse problem types, including the Discrete Fourier Transform, Monte Carlo Pi Estimation, Vector Addition, N-Body Problem, Matrix Multiplication, Convolution, and KMeans Clustering. To increase complexity, the latter three include additional parameters: kernel size (not to be confused with a CUDA kernel) for Convolution, the number of clusters for KMeans, and the tile width for matrix multiplication’s use of shared memory.

We run the experiments on four different NVIDIA devices with CUDA compute capability of at least 5.2:

- GeForce RTX 2080 Ti
- Titan V

- GeForce GTX Titan X
- GeForce RTX 4070

For each experiment, alongside the kernel launch parameters and problem size, we test with the following device parameters:

- Total Global Memory (KB)
- Multiprocessor Count
- Integrated (binary)
- Asynchronous Engine Count
- Memory Bus Width
- Memory Clock Rate (KHz)
- L2 Cache Size (bytes)
- Max Threads/SM

The dataset combines GPU configuration parameters with kernel-specific problem sizes to enable accurate prediction and optimization of execution times. Key GPU configuration features include total global memory (KB), clock rate (KHz), multiprocessor count, asynchronous engine count, memory bus width (bits), memory clock rate (KHz), L2 cache size (bytes), and the maximum number of threads per streaming multiprocessor.

Problem size is incorporated as a critical feature to capture its impact on kernel performance. For each study, the problem size defines the computational workload. Examples include matrix dimensions in Matrix Multiplication, the number of clusters defined in K-Means, or the number of bodies in the N-Body problem. Thread and block configurations, represent the parallel execution strategy.

This comprehensive integration of parameters allows the model to minimize execution time while identifying optimal configurations for each kernel, workload, and GPU setup. Through this optimization, the framework provides actionable insights into performance tuning.

C. Model Development

The predictive machine learning approach leverages a gradient-boosted decision tree model, XGBoost, to estimate CUDA kernel execution times. This model incorporates both GPU hardware configuration parameters and kernel-specific problem sizes to ensure accurate performance predictions and optimization.

The XGBoost model is trained to minimize execution time, leveraging its ability to handle complex feature interactions and rank their importance. Through this optimization, the model not only predicts execution times but also identifies the optimal thread and block configurations for specific workloads, ensuring efficient execution across varying problem sizes and GPU architectures. Dataset is split into 80% training and 20% validation.

D. Find Optimal Configuration

To identify the optimal configuration, a wide range of thread and block configurations was prepared, including configurations that were not explicitly represented in the training dataset used during model development. For a given problem size, the trained XGBoost model was applied to predict the

execution times for each configuration. By comparing the estimated execution times, the configuration yielding the lowest predicted runtime was selected as the optimal configuration. This approach enables the model to extrapolate beyond the training data, ensuring efficient execution even for untested configurations and workloads.

Since the desired output of the model is the optimal parameter configuration, not the predicted time, we define the measure of success as the normalized mean absolute error between the ten lowest predicted computation times to the actual ten lowest execution times for a given problem size. We also conduct a search of the parameter space locally around the configuration corresponding to the predicted best computation time to compare against the ground truth. To avoid becoming stuck in any local minima, our search space also includes other configurations.

E. Feature Importance

To understand the underlying factors influencing the predictive model and gain insights into kernel performance, we analyzed feature importance using XGBoost’s built-in metrics and SHAP (SHapley Additive exPlanations). XGBoost provides a ranking of features based on their contributions to the model’s decision-making process, highlighting key parameters such as thread block size, grid size, and shared memory usage as significant contributors excluding for problem size which significantly makes different execution time.

SHAP analysis was employed to provide a deeper, interpretable understanding of how each feature impacts the model’s predictions, both globally and locally. The SHAP summary plots revealed which features consistently influenced execution time across all configurations, while SHAP dependence plots uncovered interactions between features, such as the interplay between thread block size and shared memory usage. These insights allowed us to identify the most critical parameters affecting execution time and informed our understanding of how the model utilizes configuration and workload data to make predictions.

IV. RESULTS

A. Model Evaluation

To validate the model, the execution time and estimated time were normalized with respect to the problem size for each configuration. This normalization, performed using the maximum real execution time of each problem size, allowed for a direct comparison between the actual performance and the predicted performance, independent of the problem size scale. Additionally, mean aggregation was applied based on the total number of threads, combining results across varying block and thread-per-block configurations to provide a comprehensive view of performance trends. The results of 2D Convolution, visualized through a line graph (Fig. 1), highlight the differences between the normalized execution and estimated times.

Further empirical model validation resulted from the comparison of the top ten predicted configurations with the actual

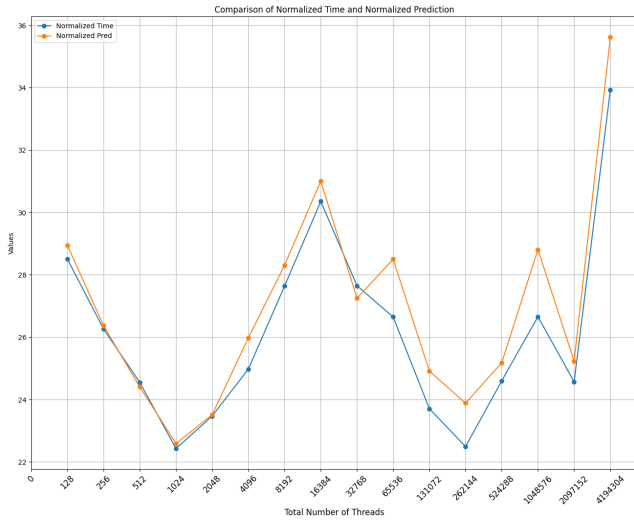


Fig. 1. Model Prediction for execution time and the difference between actual normalized execution time and normalized estimated time in a problem size.

top ten configurations. In table 1 we compare the predicted vs actual of Discrete Fourier Transform in this manner for a problem size of 100k elements.

TABLE I
DISCRETE FOURIER TRANSFORM

Table Head	Predicted			Actual		
	Blocks	TPB ^a	Time(s)	Blocks	TPB ^a	Time(s)
1	32768	32	0.280504	2048	128	0.283088
2	16384	32	0.280989	16384	32	0.283709
3	32768	128	0.283736	8192	32	0.284626
4	16384	128	0.283738	4096	32	0.284730
5	2048	128	0.284373	512	256	0.286160
6	4096	128	0.285474	16384	128	0.286175
7	4096	32	0.285739	2048	256	0.286203
8	8192	128	0.287684	8192	128	0.287351
9	8192	32	0.288904	32768	256	0.287526
10	1024	128	0.289056	8192	256	0.287867

^aTPB: Threads Per Block

TABLE II
N-BODY SIMULATION

Table Head	Predicted			Actual		
	Blocks	TPB ^a	Time(s)	Blocks	TPB ^a	Time(s)
1	4	64	2.288692	4	64	2.910407
2	16	64	2.384549	8	64	2.923885
3	1	128	2.413837	2	64	2.931870
4	2	128	2.418073	1	64	2.937146
5	1	64	2.418527	1	128	2.955249
6	8	64	2.448682	128	64	2.983417
7	2	64	2.454859	4	128	2.983581
8	16	128	2.459163	8	128	3.005980
9	32	64	2.459737	32	64	3.032347
10	8	128	2.463107	2	128	3.062799

^aTPB: Threads Per Block

From table 1, we can see significant intersection of the best ten configurations, specifically, the threads per block. This indicates that for DFT, having more blocks with fewer threads

TABLE III
PREDICTED VS ACTUAL OPTIMAL PARAMETERS

Problem Name	Predicted		Actual	
	Blocks/TPB	Time (s)	Blocks/TPB	Time (s)
Monte Carlo Pi	32,768/100	0.00051	8,192/50	0.00052
DFT ^a	32,768/32	0.28321	2,048/128	0.28308
KMeans	512/256	0.09735	64/64	0.10844
N-body	4/64	2.28869	4/64	2.91040
Matrix Mult.	611,902/16,384	0.13399	156.25M/64	0.78204
Vector Add	64/200	0.02647	512/200	0.02626
2D Conv	16/16	4315.37	26/64	4365.76

^aDFT: Discrete Fourier Transform

TABLE IV
PROBLEM METADATA

Problem Name	Problem Size	Device
Monte Carlo Pi Estimation	10 million	CUDA4
Discrete Fourier Transform	100k	CUDA4
KMeans	10 million	CUDA4
N-body Simulation	100k	CUDA2
Matrix Multiplication	100k	CUDA2
Vector Addition	10 million	CUDA4
2D Convolution	16384x16384	CUDA3

per block performs better on average than having larger block sizes. Note that to calculate the ground truth data to compare against, we pick a few example problem sizes, and take the mean of 20 iterations stratified over a large array of kernel configurations.

The SHAP framework a model-agnostic explanation tool that uses local contribution values for each data point and global aggregation to explain the overall feature importance in the model. Motivated to understand the influence of the grid sizes and block sizes individually and their impact on the prediction of the model, we employed SHAP as well as XGBoost's built-in feature importance metrics to find the 5 features that contributed most to the model's predictions. We observe the implementation for Discrete Fourier Transform:

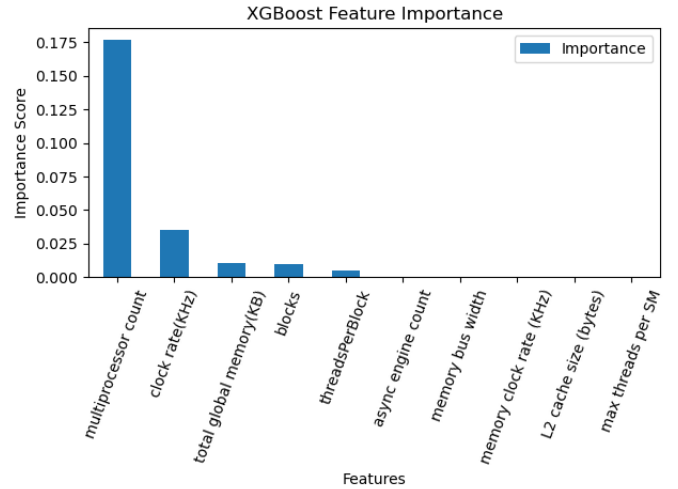


Fig. 2. XGBoost Feature Importance for Discrete Fourier Transform

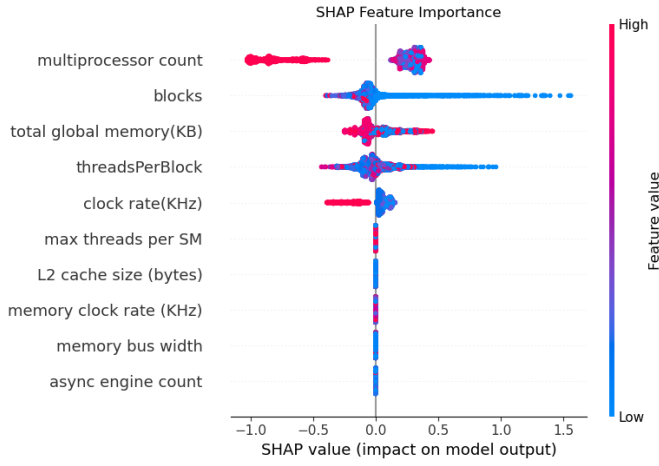


Fig. 3. SHAP Feature Importance for Discrete Fourier Transform

From our feature analysis, we observe that the hardware characteristics are the most influential on the predictions, with the amount of SM's present on the having the biggest effect on computation time. After the number of SM's the feature importance measurements disagree slightly but still find the same 5 features having the most influence, and number of blocks being slightly more influential than the number of threads per block.

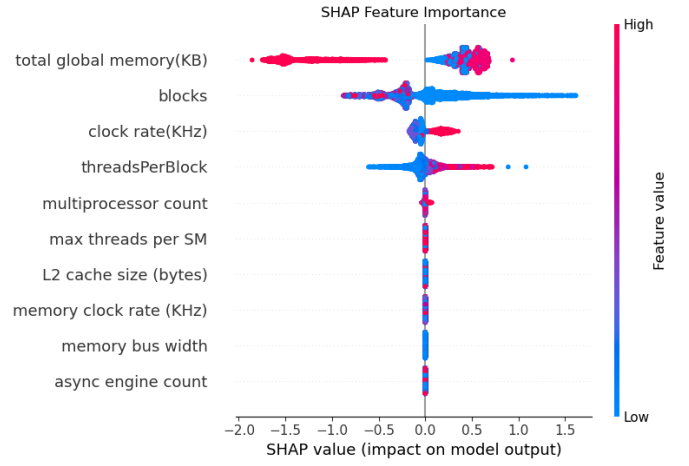


Fig. 5. SHAP Feature Importance for Monte Carlo Pi Estimation

SHAP feature explanation of Monte Carlo Pi Estimation features indicate that the amount of global memory is the biggest bottleneck in this scenario.

V. CONCLUSION

Our models demonstrated good predictive performance when trained on individual problems. We generally see the best predicted configurations aligning with multiples of the warp size (32 threads) that is universal to NVIDIA architecture. From our feature explanations using SHAP, we can observe the existence of latent variables native to each individual problem based on kernel computations, memory access patterns, and other characteristics that offer opportunities for future work, including a much more complex problem-agnostic model.

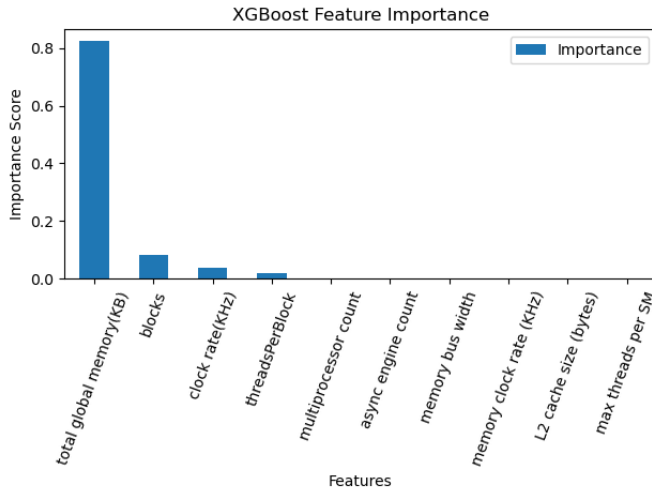


Fig. 4. XGBoost Feature Importance for Monte Carlo Pi Estimation

REFERENCES

- [1] C. Nugteren and V. Codreanu, "Cltune: A generic auto-tuner for opencl kernels," in *2015 IEEE 9th International Symposium on Embedded Multicore/Many-core Systems-on-Chip*, 2015, pp. 195–202. DOI: 10.1109/MCSoc.2015.10.
- [2] B. van Werkhoven, "Kernel tuner: A search-optimizing gpu code auto-tuner," *Future Generation Computer Systems*, vol. 90, pp. 347–358, 2019, ISSN: 0167-739X. DOI: <https://doi.org/10.1016/j.future.2018.08.004>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167739X18313359>.
- [3] J. Filipovič, F. Petrovič, and S. Benkner, "Autotuning of opencl kernels with global optimizations," in *Proceedings of the 1st Workshop on AutotuniNg and ADaptivity AppRoaches for Energy Efficient HPC Systems*, ser. ANDARE '17, Portland, OR, USA: Association for Computing Machinery, 2017, ISBN: 9781450353632. DOI: 10.1145/3152821.3152877. [Online]. Available: <https://doi.org/10.1145/3152821.3152877>.
- [4] A. Rasch, R. Schulze, M. Steuwer, and S. Gorlatch, "Efficient auto-tuning of parallel programs with inter-dependent tuning parameters via auto-tuning framework (atf)," *ACM Trans. Archit. Code Optim.*, vol. 18, no. 1, Jan. 2021, ISSN: 1544-3566. DOI: 10.1145/3427093. [Online]. Available: <https://doi.org/10.1145/3427093>.
- [5] A. Pendlebury, R. Stedmon, and C. West, "Digital control of a process using on-line auto-tuning," *IFAC Proceedings Volumes*, vol. 4, no. 1, pp. 137–143, 1971, 2nd IFAC Conference on Instrumentation and Automation in the Paper, Rubber and Plastics Industries, Brussels, Belgium, May 24-28, 1971, ISSN: 1474-6670. DOI: [https://doi.org/10.1016/S1474-6670\(17\)68702-0](https://doi.org/10.1016/S1474-6670(17)68702-0). [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1474667017687020>.
- [6] S. Suzuki and K. Fukunishi, "An auto-tuning method for control system parameters in nuclear power plants," *Nuclear Technology*, vol. 58, no. 3, pp. 379–387, 1982. DOI: 10.13182/NT82-A32973. eprint: <https://doi.org/10.13182/NT82-A32973>. [Online]. Available: <https://doi.org/10.13182/NT82-A32973>.
- [7] J. Bilmes, K. Asanovic, C.-W. Chin, and J. Demmel, "Optimizing matrix multiply using phipac: A portable, high-performance, ansi c coding methodology," in *ACM International Conference on Supercomputing 25th Anniversary Volume*, 1997, pp. 253–260.
- [8] S. Lundberg and S.-I. Lee, *A unified approach to interpreting model predictions*, 2017. arXiv: 1705.07874 [cs.AI]. [Online]. Available: <https://arxiv.org/abs/1705.07874>.
- [9] I. Baldini, S. J. Fink, and E. Altman, "Predicting gpu performance from cpu runs using machine learning," in *2014 IEEE 26th International Symposium on Computer Architecture and High Performance Computing*, 2014, pp. 254–261. DOI: 10.1109/SBAC-PAD.2014.30.
- [10] A. Karami, S. A. Mirsoleimani, and F. Khunjush, "A statistical performance prediction model for opencl kernels on nvidia gpus," in *The 17th CSI International Symposium on Computer Architecture & Digital Systems (CADSD 2013)*, IEEE, 2013, pp. 15–22.
- [11] M. Amarís, R. Y. de Camargo, M. Dyab, A. Goldman, and D. Trystram, "A comparison of gpu execution time prediction using machine learning and analytical modeling," in *2016 IEEE 15th International Symposium on Network Computing and Applications (NCA)*, 2016, pp. 326–333. DOI: 10.1109/NCA.2016.7778637.
- [12] A. Brandt, M. M. Maza, T. Jeshani, L. Wang, D. Mohajerani, and J. Paudel, "Dynamically finding optimal kernel launch parameters for cuda programs," in *Proceedings of the 33rd Annual International Conference on Computer Science and Software Engineering*, ser. CASCON '23, Las Vegas, NV, USA: IBM Corp., 2023, pp. 64–73.
- [13] T. A. Connors and A. Qasem, "Automatically selecting profitable thread block sizes for accelerated kernels," in *2017 IEEE 19th International Conference on High Performance Computing and Communications; IEEE 15th International Conference on Smart City; IEEE 3rd International Conference on Data Science and Systems (HPCC/SmartCity/DSS)*, 2017, pp. 442–449. DOI: 10.1109/HPCC-SmartCity-DSS.2017.58.
- [14] R. Lim, B. Norris, and A. Malony, "Autotuning gpu kernels via static and predictive analysis," in *2017 46th International Conference on Parallel Processing (ICPP)*, 2017, pp. 523–532. DOI: 10.1109/ICPP.2017.61.
- [15] N.-P. Tran and M. Lee, "Parameter tuning model for optimizing application performance on gpu," in *2016 IEEE 1st International Workshops on Foundations and Applications of Self* Systems (FAS* W)*, IEEE, 2016, pp. 78–83.